Reporte Técnico: Proceso de Fusión de Ramas en Git y Resolución de Errores

Historial de incidentes, análisis de fallos y resolución paso a paso en el repositorio Repo_Tecnicos

Institución / Equipo:	Universidad Tecnológica de Coahuila (UTC) | Equipo Los Técnicos
Repositorio GitHub:	https://github.com/pyedgar/Repo_Tecnicos
Ramas Involucradas:	main (principal), E_Reyes (Edgar), F_Gonzalez (Fátima)
Fecha de la Sesión:	28 de Septiembre de 2026

RESUMEN EJECUTIVO: Este documento recopila de manera formal todos los problemas técnicos y bloqueos experimentados al fusionar las ramas individuales de trabajo hacia la rama principal 'main', detallando cada causa y la solución definitiva con evidencia gráfica de la consola.

1. Introducción y Arquitectura del Repositorio

El proyecto 'Repo_Tecnicos' requería integrar los avances desarrollados de forma paralela por los integrantes del equipo en sus ramas secundarias. El flujo de trabajo previsto consistía en posicionarse en la rama 'main', sincronizar cambios remotos, fusionar las ramas 'E_Reyes' y 'F_Gonzalez', y posteriormente publicar el resultado final en GitHub mediante 'git push origin main'.

Durante esta integración, surgieron múltiples obstáculos técnicos propios del manejo de ramas en Git, estados de bloqueo local, conflictos entre archivos y editores interactivos en consola. A continuación se desglosan en orden cronológico.

2. Registro Cronológico de Incidentes y Errores

Incidente 1: Rama principal 'main' no reconocida en local

- **Comando ejecutado:** git checkout main
- **Error retornado:** error: pathspec 'main' did not match any file(s) known to git

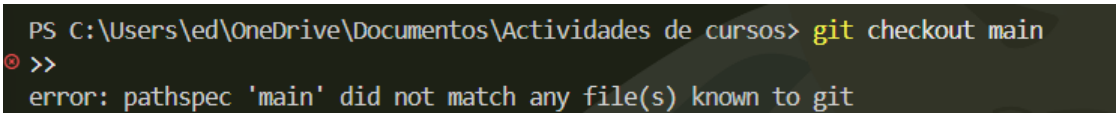


Figura 1: Fallo al intentar cambiar a la rama 'main' debido a referencias remotas no sincronizadas.

Causa Técnica: El repositorio local no contaba con la referencia de la rama 'main' porque el clon o trabajo inicial se realizó directamente sobre la rama 'E_Reyes' sin descargar el mapeo completo de ramas de GitHub.
Solución: Se ejecutó 'git fetch origin' para descargar la totalidad de referencias remotas. A partir de ese momento, Git reconoció 'origin/main' y permitió la creación y cambio a la rama local con 'git checkout main'.

```
git fetch origin
git checkout main
```

Incidente 2: Bloqueo en el editor interactivo Vim (MERGE_MSG)

• **Comando ejecutado:** git merge E_Reyes
• **Comportamiento observado:** La terminal abrió automáticamente el editor Vim solicitando confirmación de commit, mostrando: E211: File '.git/MERGE_MSG' no longer available

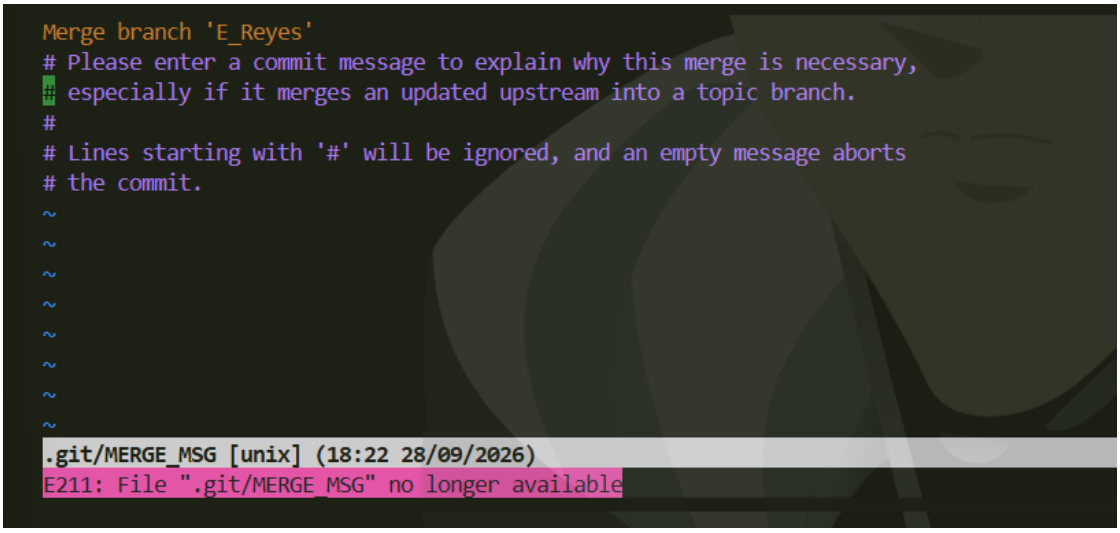


Figura 2: Editor Vim desplegado en consola requiriendo confirmar el mensaje del merge commit.

Causa Técnica: Cuando una fusión no puede resolverse como 'fast-forward', Git crea un 'Merge Commit' y exige ingresar un mensaje. En terminales Windows con Git for Windows, el editor por omisión es Vim. Si no se dominan los comandos modales de Vim, el usuario queda atrapado sin poder volver al símbolo de comando.
Solución: Se instruyó la secuencia estándar para guardar y cerrar en Vim: presionar [Esc] (modo normal), escribir ':wq' (write and quit) y presionar [Enter]. Con esto se confirmó el commit y se liberó la consola.

```
# Dentro de la pantalla de Vim:
```

```
# 1. Presionar tecla ESC
# 2. Escribir:
:wq
# 3. Presionar ENTER
```

Incidente 3: Imposibilidad de cambiar de rama durante un Merge en pausa

• **Comando ejecutado (Entorno F_Gonzalez):** git switch F_Gonzalez
• **Error retornado:** fatal: cannot switch branch while merging (Consider 'git merge --quit' or 'git worktree add')

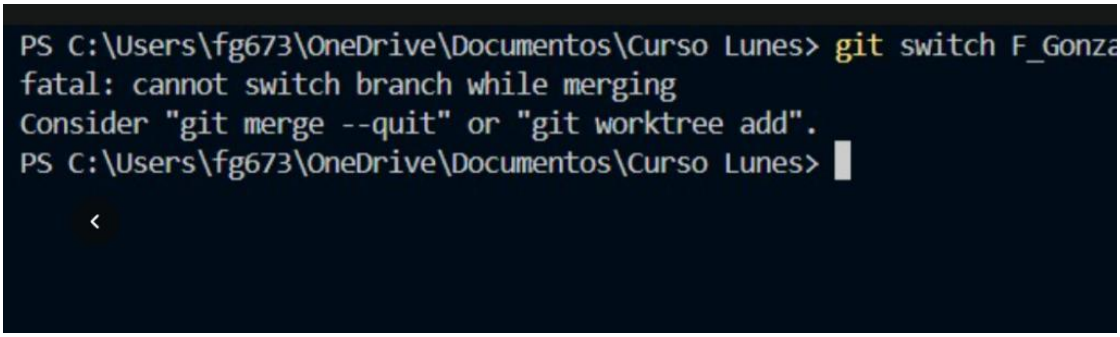


Figura 3: Bloqueo de Git al intentar alternar ramas mientras existe una fusión inconclusa.

Causa Técnica: Fátima inició un merge en su máquina pero el proceso no concluyó (debido a conflictos o falta de commit). Git entra en un estado protegido de 'MERGING', impidiendo cambiar de rama para proteger el árbol de archivos de pérdidas o inconsistencias.
Solución: Para restablecer el repositorio a su estado previo seguro, se debe abortar el merge incompleto con 'git merge --abort', o bien resolver los conflictos en los archivos afectados, añadirlos con 'git add .' y cerrar con 'git commit'.

```
# Cancelar y volver al estado limpio antes del merge:
git merge --abort

# O resolver y finalizar:
git add .
git commit -m "Resueltos conflictos de merge"
```

Incidente 4: Bloqueo de Git por archivo remanente 'index.lock'

• **Operación:** Intento de confirmación de cambios en Source Control de Visual Studio Code
• **Error retornado en ventana modal:** Git: fatal: Unable to create '.../.git/index.lock': File exists.

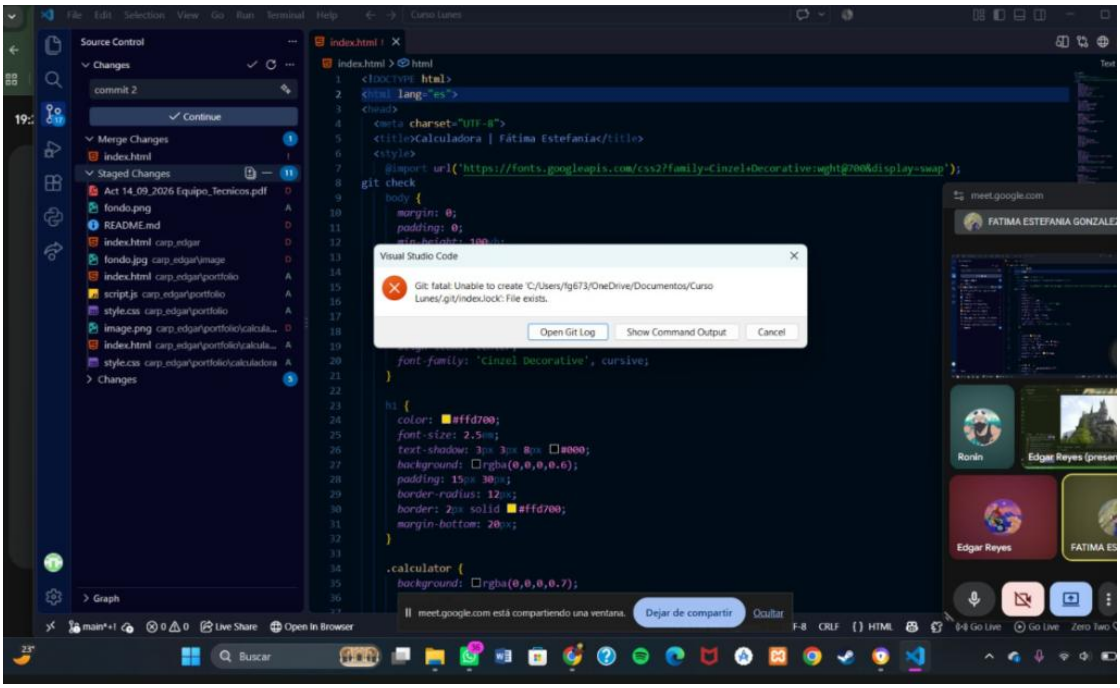


Figura 4: Diálogo de error en Visual Studio Code indicando cerrojo retenido por .git/index.lock.

Causa Técnica: Git utiliza el archivo temporal 'index.lock' como cerrojo de exclusión mutua para evitar que dos procesos modifiquen el repositorio a la vez. Cuando un proceso anterior se traba, se cancela inesperadamente o se cuelga (como ocurrió con la fusión que dejó conflictos en 'index.html'), el archivo '.lock' queda huérfano y bloquea cualquier acción futura de Git.
Solución: Eliminar manualmente el archivo de bloqueo desde la terminal de PowerShell en la carpeta del repositorio.

```
# En PowerShell de Windows:
Remove-Item ".git\index.lock" -Force

# O en Git Bash:
rm -f ".git/index.lock"
```

Incidente 5: Intento de fusionar rama inexistente en local ('not something we can merge')

• **Comando ejecutado en máquina de Edgar:** git merge F_Gonzalez
• **Error retornado:** merge: F_Gonzalez - not something we can merge
Did you mean this? origin/F_Gonzalez

Causa Técnica: La rama 'F_Gonzalez' de la compañera había sido subida exitosamente a GitHub ('origin/F_Gonzalez'), pero Edgar no la tenía creada como rama local en su computadora. Al intentar invocarla con su nombre corto, Git notificó que no era una rama local válida.
Solución: Existen dos alternativas válidas: la más rápida y limpia es fusionar directamente la rama

remota ya descargada mediante 'git merge origin/F_Gonzalez', o bien crear el rastreo local previamente mediante 'git checkout F_Gonzalez' antes de fusionar.

```
# Opción Directa (Recomendada):
git merge origin/F_Gonzalez

# Opción con clon local:
git checkout F_Gonzalez
git checkout main
git merge F_Gonzalez
```

Incidente 6: Advertencia de archivo swap de Vim (E325: ATTENTION)

• **Comportamiento observado:** Al ejecutarse la fusión de origin/F_Gonzalez, Vim se abrió con una alerta crítica: E325: ATTENTION - Found a swap file by the name '.git/.MERGE_MSG.swp'

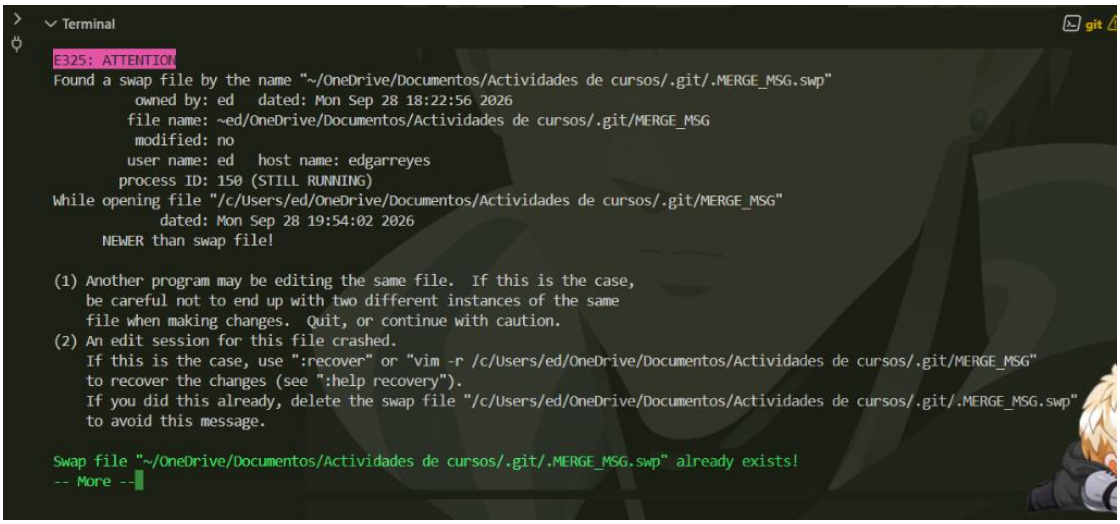


Figura 5: Pantalla de alerta E325 generada por archivo temporal swap huérfano de una sesión anterior.

Causa Técnica: Como el editor Vim anterior no finalizó con ':wq' sino que la consola se cerró forzosamente, quedó grabado el archivo '.MERGE_MSG.swp'. Al intentar editar el nuevo mensaje de commit para 'origin/F_Gonzalez', Vim detectó el archivo swap y detuvo la ejecución por seguridad.
Solución: 1. Presionar [Enter] para avanzar el aviso '-- More --'.
2. Presionar la tecla 'd' (Delete it) para eliminar el archivo swap obsoleto.
3. Entrar a la pantalla de edición, presionar [Esc], escribir ':wq' y presionar [Enter] para completar la fusión definitivamente.

```
# Pasos en la pantalla E325:
# 1. Presionar ENTER en '-- More --'
# 2. Presionar letra 'd' para eliminar el swap viejo
```

```
# 3. Presionar ESC, escribir ':wq' y presionar ENTER
```

3. Fusión Hacia Carpetas Específicas (Estrategias)

Durante el seguimiento, se solicitó una técnica para incorporar el código de una rama externa pero ubicándolo exclusivamente dentro de un subdirectorio designado, evitando mezclar archivos en la raíz del proyecto. Se presentaron dos mecanismos:

Método	Comando / Procedimiento	Ventajas y Consideraciones
Extracción Selectiva (Checkout)	git checkout <rama> -- . # Luego mover a carpeta con script o mv	Sencillo e intuitivo. No crea historial formal de fusión en Git; ideal para inspeccionar o aislar código.
Git Subtree (Merge Oficial)	1. git merge -s ours --no-commit <rama> 2. git read-tree --prefix=<carpeta>/ -u <rama> 3. git commit -m "Merge en carpeta"	Enfoque formal. Preserva el registro de Git y coloca todo el árbol de la otra rama bajo el prefijo deseado.

4. Protocolo Recomendado de Fusión (Cheat Sheet)

Para futuras colaboraciones en equipo, el siguiente protocolo garantiza una sincronización sin bloqueos ni errores de referencias:

```
# PASO 1: Descargar el estado completo del repositorio remoto
git fetch --all

# PASO 2: Situarse en la rama principal y asegurar paridad con GitHub
git checkout main
git pull origin main

# PASO 3: Fusionar directamente la rama del compañero desde origin
git merge origin/Nombre_De_La_Rama

# PASO 4: En caso de que se abra Vim para confirmar mensaje de merge:
#   1. Pulsar [Esc]
#   2. Escribir :wq
#   3. Pulsar [Enter]

# PASO 5: En caso de que ocurra bloqueo por index.lock huérfano:
#   PowerShell: Remove-Item ".git\index.lock" -Force
```

```
# PASO 6: Publicar los cambios consolidados en GitHub
git push origin main
```